

Systeme Linux

Chapitre 1 : Introduction à Linux et son architecture (5H)

I. Introduction à Linux

1. Qu'est-ce que Linux ?

Linux est un système d'exploitation open source basé sur le noyau Linux, qui a été créé par **Linus Torvalds** en 1991. Il est utilisé dans une grande variété d'environnements, allant des serveurs aux ordinateurs de bureau, en passant par les smartphones (Android est basé sur Linux), les objets connectés et même les superordinateurs.

2. Caractéristiques principales de Linux :

Open source : Son code source est librement accessible et modifiable.
Multi-utilisateur et multitâche : Plusieurs utilisateurs peuvent travailler simultanément, et plusieurs tâches peuvent être exécutées en parallèle.
Sécurisé et stable : Il est réputé pour sa robustesse et sa résistance aux virus et aux pannes.
Personnalisable : Il existe de nombreuses **distributions Linux** adaptées à différents usages (ex. : Ubuntu, Debian, Fedora, Arch Linux).
Gratuit : Contrairement à Windows ou macOS, la majorité des distributions Linux sont gratuites.

3. Historique et évolution de Linux

L'histoire de Linux commence dans les années 1990 et s'inscrit dans la continuité des systèmes UNIX. Voici son évolution :

a. Les origines (avant 1991) : UNIX et MINIX

UNIX (1969-1970) : Créé par **Ken Thompson** et **Dennis Ritchie** chez AT&T Bell Labs, UNIX est un système d'exploitation multitâche et multi-utilisateur qui influence tous les OS modernes.
MINIX (1987) : **Andrew S. Tanenbaum** développe MINIX, un clone simplifié d'UNIX destiné à l'enseignement. Il inspire Linus Torvalds, mais son code reste sous une licence restrictive.

b. La naissance de Linux (1991)

1991 : **Linus Torvalds**, étudiant finlandais en informatique, crée son propre noyau inspiré de MINIX, car il trouvait ce dernier limité. Le **25 août 1991**, il annonce sur un forum (comp.os.minix) son projet d'un "petit OS libre". Il publie la première version du **noyau Linux (version 0.01)** en **septembre 1991** sous licence propriétaire.

c. L'essor du mouvement open source (1992-2000)

1992 : Linus publie Linux sous la **licence GNU GPL** (General Public License), ce qui permet à toute la communauté de contribuer.
1993 : Plus de **100 développeurs** travaillent sur Linux. Les premières **distributions Linux** apparaissent :

- **Slackware (1993)**
- **Debian (1993)** (créée par Ian Murdock)
1996 : Linux 2.0 est publié, avec une meilleure gestion des architectures multiprocesseurs.
Fin des années 90 : Linux devient populaire pour les **serveurs**, en concurrence avec Windows NT.

d. L'adoption massive et la diversification (2000-2010)

2000 : IBM annonce un investissement massif dans Linux.
2004 : **Ubuntu** est lancé par Canonical, rendant Linux plus accessible aux utilisateurs grand public.
2007 : Android, basé sur Linux, est développé par Google et devient plus tard le leader du marché mobile.

e. Linux aujourd'hui et son avenir (2010 - présent)

Linux est le **noyau** le plus utilisé dans le monde :

- **90% des serveurs cloud** (AWS, Google Cloud, Azure) tournent sous Linux.
 - **100% des supercalculateurs** utilisent Linux.
 - **Android (Linux-based)** domine le marché mobile.
 - **IoT, embarqué, automobile** (Tesla, systèmes avioniques) utilisent Linux.
- Évolution continue** avec des versions régulières du noyau, des améliorations en sécurité et en performance.

II. Architecture d'un système Linux : noyau, shell, applications.

L'architecture d'un système Linux est structurée pour permettre une interaction efficace entre le matériel, le noyau, le shell et les applications. Cette séparation du système facilite la gestion, la maintenance et l'évolution du système. Le système Linux se divise principalement en trois couches : le noyau, le shell et les applications. Voici chaque composant un aperçu :

1. Noyau (Kernel)

Le noyau est le cœur du système d'exploitation. Il gère les ressources matérielles et fournit des services essentiels. Ses principales fonctions incluent :

- **Gestion de la mémoire** : Allocation et libération de mémoire.
- **Gestion des processus** : Création, planification et terminaison des processus.
- **Gestion des périphériques** : Interaction avec le matériel via des pilotes.
- **Gestion des systèmes de fichiers** : Accès et organisation des données sur les disques.

2. Shell

Le shell est une interface utilisateur qui permet d'interagir avec le noyau. Il peut être de type graphique (GUI) ou en ligne de commande (CLI). Les principales fonctionnalités du shell incluent :

- **Interprétation des commandes** : Les utilisateurs saisissent des commandes qui sont interprétées et exécutées par le noyau.
- **Scripting** : Possibilité d'automatiser des tâches via des scripts.
- **Gestion des entrées/sorties** : Redirection et pipe pour manipuler les flux de données.

3. Applications

Les applications sont les programmes qui s'exécutent sur le système d'exploitation. Elles peuvent être de divers types :

- **Applications de bureau** : Navigateurs, éditeurs de texte, outils de bureautique.
- **Utilitaires système** : Outils de gestion de fichiers, de réseau, de sécurité.
- **Applications serveur** : Serveurs web, bases de données, applications réseau.

III. Différences entre Linux et autres systèmes d'exploitation (Windows, macOS)

Caractéristiques	Linux	Windows	macOS
Origine	Basé sur UNIX (inspiré de MINIX)	Développé par Microsoft	Basé sur UNIX (BSD) et développé par Apple
Licence	Open source (GNU GPL, etc.)	Propriétaire	Propriétaire
Coût	Gratuit (la plupart des distributions)	Payant (inclus dans le prix du PC ou de la licence)	Payant (inclus dans les appareils Apple)
Sécurité	Très sécurisé, peu de virus	Plus vulnérable aux virus et attaques	Sécurisé, mais cible de certaines attaques
Personnalisation	Très personnalisable (choix d'environnements graphiques, terminal, etc.)	Personnalisation limitée	Personnalisation restreinte
Compatibilité logicielle	Compatible avec la plupart des logiciels open source, mais moins avec les logiciels propriétaires (Adobe, Microsoft Office)	Large compatibilité avec les logiciels commerciaux	Compatible avec l'écosystème Apple, moins de logiciels tiers
Utilisation principale	Serveurs, développement, cybersécurité, systèmes embarqués, supercalculateurs	Bureautique, gaming, usage général	Design, multimédia, production vidéo/musique
Performance et légèreté	Peut être optimisé pour de vieux PC ou des machines performantes	Peut devenir lent avec le temps à cause des mises à jour	Optimisé pour le matériel Apple, fluide mais lourd
Système de fichiers	ext4, XFS, Btrfs	NTFS, FAT32, exFAT	APFS, HFS+

Caractéristiques	Linux	Windows	macOS
Gestion des mises à jour	Contrôle total, mises à jour optionnelles	Mises à jour automatiques parfois imposées	Mises à jour fréquentes et intégrées au système
Support matériel	Large support, mais certains pilotes propriétaires peuvent manquer	Compatible avec la majorité du matériel	Optimisé uniquement pour les appareils Apple
Interface utilisateur	Variée selon l'environnement de bureau (GNOME, KDE, XFCE, etc.)	Interface unique et standardisée	Interface fluide et soignée

Les différences majeures :

Linux : Flexible, sécurisé, idéal pour les développeurs et serveurs.

Windows : Grand public, gaming, bureautique, mais plus vulnérable aux virus.

macOS : Fluide, optimisé pour la créativité, mais restreint à l'écosystème Apple

IV. Les principales distributions Linux (Debian, Ubuntu, Red Hat, CentOS)

1. Qu'est-ce qu'une distribution ?

Une distribution Linux (ou "distro") est un ensemble de logiciels bâtis autour du noyau Linux, incluant des outils, des bibliothèques et des applications.

2. Les principales distributions Linux et leurs caractéristiques

Il existe de nombreuses distributions Linux, adaptées à différents usages. Voici un aperçu des principales :

a. Debian

Présentation : Debian est l'une des plus anciennes distributions Linux (créée en 1993) et sert de base à de nombreuses autres distributions.

Caractéristiques :

- Très **stable** et sécurisée.
- Large **communauté** et nombreux logiciels disponibles.
- Utilise un gestionnaire de paquets **.deb** (APT).
Idéal pour : Serveurs, utilisateurs avancés, entreprises recherchant une stabilité maximale.

b. Ubuntu (Basée sur Debian)

Présentation : Développée par Canonical en 2004, Ubuntu est l'une des distributions les plus populaires et faciles à utiliser.

Caractéristiques :

- Interface utilisateur conviviale (**GNOME** par défaut).
- Grande compatibilité logicielle et support matériel.
- Versions **LTS (Long Term Support)** avec support de 5 ans.
Idéal pour : Débutants, ordinateurs de bureau, serveurs, développement.

c. Red Hat Enterprise Linux (RHEL)

Présentation : Red Hat Enterprise Linux est une distribution payante et supportée commercialement, principalement utilisée en entreprise.

Caractéristiques :

- Très stable, optimisée pour les **grandes entreprises et data centers**.
- Utilise un gestionnaire de paquets **.rpm** (DNF/YUM).
- Support technique professionnel et mises à jour de sécurité.
Idéal pour : Entreprises, infrastructures critiques, cloud computing.

d. CentOS (Dérivée de RHEL - Discontinué en 2021)

Présentation : CentOS était une version communautaire de RHEL, offrant une alternative gratuite et stable pour les serveurs.

Caractéristiques :

- Identique à RHEL, mais sans support commercial.
- Très utilisée pour les serveurs et hébergements web.
Statut actuel : **Discontinué en 2021** au profit de **CentOS Stream**, une version en développement continu moins stable.
Alternative : **AlmaLinux** et **Rocky Linux**, qui reprennent le rôle de CentOS.

Résumé rapide

Distribution	Basée sur	Public cible	Gestion des paquets
Debian	Indépendante	Serveurs, utilisateurs avancés	.deb (APT)
Ubuntu	Debian	Débutants, bureautique, serveurs	.deb (APT)
RHEL	Indépendante	Entreprises, cloud	.rpm (DNF/YUM)
CentOS (discontinué)	RHEL	Serveurs, entreprises	.rpm (DNF/YUM)

Impact des distributions

Les différentes distributions ont permis l'adoption de Linux dans divers secteurs, rendant le système accessible à un large éventail d'utilisateurs.

V. Processus d'installation de Linux et configuration initiale

L'installation de Linux et la configuration initiale impliquent plusieurs étapes clés. Voici un guide général :

1. Choix de la Distribution

- **Sélectionner une distribution** : Debian, Ubuntu, Red Hat, CentOS, etc.
- **Télécharger l'image ISO** : Depuis le site officiel de la distribution choisie.

2. Création d'un Support d'Installation

- **Graver l'ISO sur un DVD** ou **créer une clé USB bootable** avec un outil comme Rufus ou balenaEtcher.

3. Démarrage à partir du Support d'Installation

- **Insérer le support** dans l'ordinateur.
- **Redémarrer l'ordinateur** et accéder au menu de démarrage (souvent en appuyant sur F12, Esc, ou une autre touche).
- **Sélectionner le support d'installation** pour démarrer l'installation de Linux.

4. Installation de Linux

- **Sélectionner la langue** et les options régionales.
- **Choisir le type d'installation** : Installation standard ou avancée (partitionnement manuel).
- **Partitionnement du disque** :
 - **Guidé** : L'installateur crée des partitions automatiquement.
 - **Manuel** : Créer des partitions spécifiques (root, swap, home).
- **Installer le système** : Suivre les instructions à l'écran pour installer Linux.

5. Configuration Initiale

- **Créer un utilisateur** : Choisir un nom d'utilisateur et un mot de passe.
- **Configurer le réseau** : Connexion Wi-Fi ou Ethernet.
- **Mettre à jour le système** : Installer les mises à jour disponibles après l'installation.

6. Installation de Logiciels Supplémentaires

- **Utiliser le gestionnaire de paquets** : Installer des logiciels supplémentaires selon vos besoins (navigateur, outils de développement, etc.).

7. Personnalisation de l'Environnement

- **Configurer le bureau** : Choisir des thèmes, arrière-plans, et disposition des icônes.
- **Ajouter des extensions ou des widgets** : Selon la distribution et l'environnement de bureau.

8. Sauvegarde et Sécurité

- **Configurer des sauvegardes régulières** : Utiliser des outils comme Deja Dup ou rsync.
- **Configurer un pare-feu** : Utiliser ufw ou d'autres outils pour sécuriser le système.

Chapitre 2 : Commandes de base et gestion des fichiers (3H)

I. Navigation dans le système de fichiers (cd, ls, pwd)

Naviguer dans le système de fichiers sous Linux est essentiel pour gérer les fichiers et répertoires. Voici un aperçu des commandes de base : `cd`, `ls`, et `pwd`.

1. pwd (Print Working Directory)

- **Description** : Affiche le chemin absolu du répertoire courant.
- **Utilisation** :

```
pwd
```

Exemple :

```
/home/utilisateur
```

2. ls (List)

- **Description** : Liste les fichiers et répertoires dans le répertoire courant.
- **Options courantes** :
 - `-l` : Affiche des détails (permissions, propriétaire, taille, date).
 - `-a` : Affiche tous les fichiers, y compris ceux qui commencent par un point (fichiers cachés).
- **Utilisation** :

```
ls
ls -l
ls -a
```

Exemple :

```
fichier1.txt dossier1 .fichier_cache
```

3. cd (Change Directory)

- **Description** : Change le répertoire courant.
- **Utilisation** :
 - Pour aller à un répertoire spécifique :

```
cd nom du repertoire
```

Pour revenir au répertoire parent :

```
cd ..
```

Pour retourner au répertoire personnel :

```
cd ~
```

Exemple :

```
cd Documents
```

Exemples de Navigation

1. Vérifiez votre répertoire courant :

```
pwd
```

Listez les fichiers dans le répertoire courant :


```
ls
```

Accédez à un sous-répertoire :

```
cd sous_dossier
```

Revenez au répertoire parent :

```
cd ..
```

Ces commandes vous permettent de naviguer efficacement dans le système de fichiers Linux. Familiarisez-vous avec elles pour gérer vos fichiers et répertoires de manière fluide.

II. Manipulation des fichiers et des répertoires (cp, mv, rm, mkdir)

Voici un aperçu des commandes de base pour la manipulation des fichiers et des répertoires sous Linux : cp, mv, rm et mkdir.

1. cp (Copy)

- **Description** : Copie des fichiers ou des répertoires.
- **Utilisation** :
 - Pour copier un fichier :

```
cp source.txt destination.txt
```

Pour copier un répertoire (avec l'option `-r` pour la récursivité) :

```
cp -r dossier/ destination_dossier/
```

Exemple :

```
cp fichier1.txt dossier_copie/
```

2. mv (Move)

- **Description** : Déplace ou renomme des fichiers ou des répertoires.
- **Utilisation** :
 - Pour déplacer un fichier :

```
mv fichier.txt nouveau_dossier/
```

Pour renommer un fichier :

```
mv ancien_nom.txt nouveau_nom.txt
```

Exemple :

```
mv fichier1.txt dossier_destination/
```

3. rm (Remove)

- **Description** : Supprime des fichiers ou des répertoires.
- **Utilisation** :
 - Pour supprimer un fichier :

```
rm fichier.txt
```

Pour supprimer un répertoire (avec l'option `-r` pour la récursivité) :

```
rm -r dossier/
```

Pour forcer la suppression sans confirmation :

```
rm -f fichier.txt
```

Exemple :

```
rm fichier_a_supprimer.txt
```

4. mkdir (Make Directory)

- **Description :** Crée un nouveau répertoire.
- **Utilisation :**

```
mkdir nom du dossier
```

Exemple :

```
mkdir nouveau dossier
```

Exemples de Manipulation

1. Copier un fichier :

```
cp document.txt sauvegarde document.txt
```

Déplacer un fichier dans un autre répertoire :

```
mv rapport.txt /home/utilisateur/Documents/
```

Supprimer un fichier :

```
rm ancien_document.txt
```

Créer un nouveau répertoire :

```
mkdir projets
```

Ces commandes sont essentielles pour la gestion des fichiers et des répertoires sous Linux.

III. Permissions des fichiers et gestion des utilisateurs (chmod, chown)

La gestion des permissions des fichiers et des utilisateurs sous Linux est essentielle pour la sécurité et le contrôle d'accès. Voici un aperçu des commandes principales : `chmod` et `chown`.

1. Permissions des Fichiers

Chaque fichier et répertoire a des permissions qui déterminent qui peut lire, écrire ou exécuter le fichier. Les permissions sont généralement affichées sous la forme suivante :

```
-rwxr-xr--
```

- **r** : permission de lecture (read)
- **w** : permission d'écriture (write)
- **x** : permission d'exécution (execute)
- Les trois groupes de trois lettres représentent respectivement :
 - **Propriétaire** (user)
 - **Groupe** (group)
 - **Autres** (others)

Les permissions des fichiers sous Linux peuvent être représentées par des chiffres en utilisant un système octal. Voici l'équivalence :

- **r** (read) : 4
- **w** (write) : 2
- **x** (execute) : 1

Combinaisons

Les permissions peuvent être combinées en additionnant les valeurs :

- **rwX** (lecture, écriture, exécution) : $4 + 2 + 1 = 7$
- **rw-** (lecture, écriture) : $4 + 2 + 0 = 6$
- **r-x** (lecture, exécution) : $4 + 0 + 1 = 5$
- **r--** (lecture seulement) : $4 + 0 + 0 = 4$
- **-wX** (écriture, exécution) : $0 + 2 + 1 = 3$
- **-w-** (écriture seulement) : $0 + 2 + 0 = 2$
- **--x** (exécution seulement) : $0 + 0 + 1 = 1$
- **---** (aucune permission) : $0 + 0 + 0 = 0$

Représentation Complète

Les permissions d'un fichier ou d'un répertoire sont généralement représentés par trois chiffres :

- **Premier chiffre** : permissions du propriétaire
- **Deuxième chiffre** : permissions du groupe
- **Troisième chiffre** : permissions des autres

Exemple :

- **rwXr-xr--** correspond à **755** en notation octale :
 - Propriétaire : **rwX** = 7
 - Groupe : **r-x** = 5
 - Autres : **r--** = 4

Résumé

Voici un tableau récapitulatif :

Symboles Valeur

rwX	7
rw-	6
r-x	5
r--	4
-wX	3
-w-	2
--x	1

Symboles Valeur

--- 0

2. chmod (Change Mode)

- **Description** : Modifie les permissions d'un fichier ou d'un répertoire.
- **Utilisation** :
 - Pour ajouter une permission :

```
chmod +x fichier.txt # Ajoute la permission d'exécution
```

Pour supprimer une permission :

```
chmod -w fichier.txt # Enlève la permission d'écriture
```

Pour définir des permissions spécifiques :

```
chmod 755 fichier.txt # Définit rwxr-xr-x
```

Exemple :

```
chmod 644 document.txt # Définit rw-r--r--
```

3. chown (Change Owner)

- **Description** : Change le propriétaire et/ou le groupe d'un fichier ou d'un répertoire.
- **Utilisation** :
 - Pour changer le propriétaire :

```
chown nouvel_utilisateur fichier.txt
```

Pour changer le propriétaire et le groupe :

```
chown nouvel_utilisateur:nouveau_groupe fichier.txt
```

Exemple :

```
chown alice:users document.txt # Change le propriétaire à alice et le groupe à users
```

Exemples de Gestion des Permissions et des Utilisateurs

Ajouter la permission d'exécution à un fichier :

```
chmod +x script.sh
```

Enlever la permission d'écriture à tous les utilisateurs :

```
chmod a-w fichier.txt
```

Changer le propriétaire d'un fichier :

```
chown bob fichier.txt
```

Changer le propriétaire et le groupe d'un répertoire :

```
chown -R alice:developpeurs projet/
```

La compréhension des permissions et des commandes `chmod` et `chown` est cruciale pour maintenir la sécurité et la gestion des fichiers dans un système Linux. Familiarisez-vous avec ces commandes pour mieux contrôler l'accès aux ressources de votre système.

IV. Recherche de fichiers (find, grep, locate)

Voici un aperçu des commandes essentielles pour la recherche de fichiers sous Linux : `find`, `grep`, et `locate`.

1. find

- **Description** : Recherche des fichiers et des répertoires dans un arbre de répertoires.
- **Syntaxe de base** :

```
find [chemin] [options] [expression]
```

Exemples :

Trouver tous les fichiers dans un répertoire :

```
find /chemin/du/répertoire
```

Trouver un fichier par nom :

```
find /chemin/ -name "fichier.txt"
```

Trouver tous les fichiers avec une extension spécifique :

```
find /chemin/ -name "*.jpg"
```

Trouver des fichiers plus grands qu'une certaine taille :

```
find /chemin/ -size +10M
```

2. grep

- **Description** : Recherche du texte dans les fichiers. Il peut être utilisé pour filtrer les résultats de `find`.
- **Syntaxe de base** :

```
grep [options] "texte" [fichiers]
```

Exemples :

Rechercher un mot dans un fichier :

```
grep "mot" fichier.txt
```

Rechercher récursivement dans un répertoire :

```
grep -r "mot" /chemin/du/répertoire
```

Ignorer la casse :

```
grep -i "mot" fichier.txt
```

Afficher les numéros de ligne :

```
grep -n "mot" fichier.txt
```

3. locate

- **Description** : Recherche des fichiers en utilisant une base de données pré-construite, ce qui la rend très rapide.
- **Prérequis** : L'outil `mlocate` doit être installé et la base de données mise à jour régulièrement (généralement via une tâche cron).

- **Syntaxe de base :**

```
locate [nom_du_fichier]
```

Exemples :

Trouver un fichier par son nom :

```
locate fichier.txt
```

Rechercher tous les fichiers avec une extension spécifique :

```
locate "*.jpg"
```

- **find** : Recherche dans un système de fichiers, très flexible et puissant.
- **grep** : Recherche dans le contenu des fichiers, idéal pour filtrer et trouver des lignes spécifiques.
- **locate** : Recherche rapide basée sur une base de données, efficace pour retrouver des fichiers par leur nom.

V. Compression et archivage des fichiers (tar, gzip, zip).

Voici un aperçu des commandes essentielles pour la compression et l'archivage des fichiers sous Linux : tar, gzip, et zip.

1. tar (Tape Archive)

- **Description** : Utilisé pour créer et manipuler des archives de fichiers. Il peut regrouper plusieurs fichiers en un seul fichier d'archive.
- **Création d'une archive** :

```
tar -cvf archive.tar /chemin/du/répertoire
```

- **-c** : Créer une nouvelle archive.
- **-v** : Mode verbeux (affiche les fichiers en cours d'archivage).
- **-f** : Spécifie le nom du fichier d'archive.

Extraction d'une archive :

```
tar -xvf archive.tar
```

-x : Extraire les fichiers de l'archive.

Création d'une archive compressée avec gzip :

```
tar -czvf archive.tar.gz /chemin/du/répertoire
```

-z : Comprime l'archive avec gzip.

Extraction d'une archive compressée :

```
tar -xzvf archive.tar.gz
```

2. gzip (GNU Zip)

- **Description** : Utilisé pour compresser des fichiers individuels. Il remplace le fichier original par une version compressée.
- **Compression d'un fichier** :

```
gzip fichier.txt
```

- Cela crée `fichier.txt.gz` et supprime `fichier.txt`.

Décompression d'un fichier :

```
gzip -d fichier.txt.gz
```

- Cela restaure `fichier.txt`.

3. zip

- **Description** : Utilisé pour créer des fichiers zip, qui sont des archives compressées pouvant contenir plusieurs fichiers et répertoires.
- **Création d'un fichier zip** :

```
zip archive.zip fichier1.txt fichier2.txt
```

Pour archiver un répertoire :

```
zip -r archive.zip /chemin/du/répertoire
```

`-r` : Archive récursive.

Décompression d'un fichier zip :

```
unzip archive.zip
```

- **tar** : Idéal pour archiver et compresser des répertoires complets.
- **gzip** : Spécialisé dans la compression de fichiers individuels.
- **zip** : Utilisé pour créer des archives compressées qui peuvent contenir plusieurs fichiers et sous-répertoires.

Chapitre 3 : Gestion des utilisateurs et des groupes (3H)

I. Création et gestion des comptes utilisateurs (adduser, usermod)

La création et la gestion des comptes utilisateurs sous Linux sont essentielles pour l'administration système. Voici un aperçu des commandes `adduser` et `usermod`.

1. adduser

- **Description** : Crée un nouvel utilisateur et configure son environnement.
- **Syntaxe** :

```
adduser [options] nom_utilisateur
```

Exemples :

- **Créer un nouvel utilisateur :**

```
sudo adduser alice
```

- Cela créera un utilisateur nommé `alice`, créera un répertoire personnel (`/home/alice`), et vous demandera de définir un mot de passe et d'autres informations.

- **Options courantes :**

- `--home` : Spécifie un répertoire personnel différent.
- `--shell` : Définit le shell par défaut.

2. usermod

- **Description :** Modifie un compte utilisateur existant.
- **Syntaxe :**

```
usermod [options] nom_utilisateur
```

Exemples :

Changer le mot de passe d'un utilisateur :

```
sudo passwd alice
```

Ajouter un utilisateur à un groupe :

```
sudo usermod -aG groupe alice
```

`-aG` : Ajoute l'utilisateur à un groupe sans le retirer des autres groupes.

Changer le répertoire personnel d'un utilisateur :

```
o sudo usermod -d /nouveau/chemin/alice alice
```

- **Options courantes :**

- `-l` : Change le nom d'utilisateur.
- `-d` : Change le répertoire personnel.
- `-s` : Change le shell par défaut.

Exemples de Gestion des Utilisateurs

Créer un nouvel utilisateur :

```
sudo adduser bob
```

Ajouter l'utilisateur bob au groupe sudo :

```
sudo usermod -aG sudo bob
```

Changer le shell de alice à /bin/bash :

```
sudo usermod -s /bin/bash alice
```

Changer le nom d'utilisateur de alice à alice_new :

```
sudo usermod -l alice_new alice
```

Les commandes `adduser` et `usermod` sont essentielles pour la gestion des comptes utilisateurs sous Linux.

II. Gestion des permissions et des droits d'accès (sudo, /etc/passwd).

La gestion des permissions et des droits d'accès sous Linux est cruciale pour la sécurité du système. Voici un aperçu des commandes sudo et du fichier /etc/passwd.

1. sudo

- **Description** : Permet à un utilisateur autorisé d'exécuter des commandes avec les privilèges d'un autre utilisateur, généralement l'utilisateur root.
- **Utilisation de base** :

```
sudo commande
```

Exemples :

Exécuter une commande en tant que root :

```
sudo apt update
```

Accéder à un shell root :

```
sudo -i
```

- **Configuration** :
 - Les permissions sudo sont gérées dans le fichier /etc/sudoers, où il est possible de définir quels utilisateurs peuvent exécuter quelles commandes.

2. Fichier /etc/passwd

- **Description** : Contient des informations sur les comptes utilisateurs du système. Chaque ligne représente un utilisateur.
- **Format d'une ligne** :

```
nom_utilisateur:mot_de_passe:UID:GID:description:répertoire_personnel:shell
```

- **nom_utilisateur** : Le nom de l'utilisateur.
- **mot_de_passe** : Habituellement un x (indiquant que le mot de passe est stocké dans /etc/shadow).
- **UID** : Identifiant unique de l'utilisateur.
- **GID** : Identifiant de groupe principal de l'utilisateur.
- **description** : Informations supplémentaires sur l'utilisateur.
- **répertoire_personnel** : Chemin vers le répertoire personnel de l'utilisateur.
- **shell** : Le shell par défaut de l'utilisateur.

Exemple de ligne :

```
alice:x:1001:1001:Alice User:/home/alice:/bin/bash
```

Gestion des Permissions

- **Permissions de fichier** : Chaque fichier et répertoire a des permissions qui déterminent qui peut lire, écrire ou exécuter.
- **Commandes** :

- `chmod` : Modifier les permissions.
- `chown` : Changer le propriétaire d'un fichier.

Exemples de Gestion avec `sudo` et `/etc/passwd`

1. Modifier un fichier système avec `sudo` :

```
sudo nano /etc/hosts
```

Afficher le contenu de `/etc/passwd` :

```
cat /etc/passwd
```

Ajouter un utilisateur avec des droits `sudo` :

```
sudo adduser bob
sudo usermod -aG sudo bob
```

La gestion des permissions et des droits d'accès sont essentielles pour la sécurité d'un système Linux. Utilisez `sudo` pour exécuter des commandes avec des privilèges élevés et le fichier `/etc/passwd` pour comprendre les comptes utilisateurs.

III. Gestion des groupes et des privilèges associés.

La gestion des groupes et des privilèges associés sous Linux est essentielle pour organiser les utilisateurs et contrôler l'accès aux ressources. Les groupes permettent de regrouper des utilisateurs ayant des permissions similaires. Cela simplifie la gestion des accès.

1. Commandes de gestion des groupes

a. `groupadd`

- **Description** : Crée un nouveau groupe.
- **Syntaxe** :

```
sudo groupadd nom_groupe
```

Exemple :

```
sudo groupadd developpeurs
```

b. `groupdel`

Description : Supprime un groupe existant.

Syntaxe :

```
sudo groupdel nom_groupe
```

Exemple :

```
sudo groupdel developpeurs
```

c. `usermod`

Description : Modifie un utilisateur existant, y compris son appartenance à des groupes.

Syntaxe :

```
sudo usermod -aG nom_groupe nom_utilisateur
```

Exemple :

```
sudo usermod -aG developpeurs alice
```

-aG ajoute `alice` au groupe `developpeurs` sans la retirer d'autres groupes.

d. groups

Description : Affiche les groupes auxquels un utilisateur appartient.

Syntaxe :

```
groups nom_utilisateur
```

Exemple :

```
groups alice
```

2. Fichier /etc/group

Description : Contient des informations sur les groupes. Chaque ligne représente un groupe.

Format d'une ligne :

```
nom_groupe:mot_de_passe:GID:liste_utilisateurs
```

- **nom_groupe** : Le nom du groupe.
- **mot_de_passe** : Généralement un `x` (indiquant que le mot de passe est désactivé).
- **GID** : Identifiant unique du groupe.
- **liste_utilisateurs** : Utilisateurs membres du groupe, séparés par des virgules.

Exemple de ligne :

```
developpeurs:x:1002:alice,bob
```

3. Gestion des privilèges

- Les privilèges sont souvent gérés par l'appartenance à des groupes :
 - **Groupes de système** : Par exemple, le groupe `sudo` permet aux utilisateurs d'exécuter des commandes avec des privilèges élevés.
 - **Permissions sur les fichiers** : Par exemple, un groupe peut avoir la permission de lecture, écriture ou exécution sur certains fichiers ou répertoires.

Exemples pratiques

1. Créer un groupe et ajouter des utilisateurs :

```
sudo groupadd administrateurs
sudo usermod -aG administrateurs alice
sudo usermod -aG administrateurs bob
```

Afficher les groupes d'un utilisateur :

```
groups bob
```

Supprimer un groupe :

```
sudo groupdel administrateurs
```

IV. Surveillance des connexions et des utilisateurs actifs (who, w, last).

La gestion des groupes et des privilèges associés est essentielle pour une administration efficace et sécurisée des utilisateurs sous Linux. Utilisez ces commandes pour organiser les utilisateurs et contrôler l'accès aux ressources du système.

La surveillance des connexions et des utilisateurs actifs sous Linux est essentielle pour la sécurité et la gestion du système.

1. who

- **Description** : Affiche une liste des utilisateurs actuellement connectés au système.
- **Syntaxe** :

```
who [options]
```

Exemple :

```
who
```

- Cela affichera les informations suivantes pour chaque utilisateur connecté : nom d'utilisateur, terminal, date et heure de la connexion, et adresse IP (le cas échéant).

2. w

- **Description** : Montre qui est connecté et ce qu'ils font. Fournit des informations supplémentaires par rapport à `who`.
- **Syntaxe** :

```
w [options]
```

Exemple :

```
w
```

- Les informations affichées incluent :
 - Nom d'utilisateur
 - TTY (terminal)
 - Heure de connexion
 - Temps d'inactivité
 - Charge système
 - Commande en cours d'exécution

3. last

- **Description** : Affiche l'historique des connexions des utilisateurs. Il lit les informations du fichier `/var/log/wtmp`.
- **Syntaxe** :

```
last [options] [utilisateur]
```

Exemple :

```
last
```

- Cela affichera une liste des utilisateurs qui se sont connectés et déconnectés, avec les dates et heures des connexions.

Pour afficher les connexions d'un utilisateur spécifique :

```
last alice
```

Options Utiles

- **who** :
 - `-u` : Affiche le temps d'inactivité de chaque utilisateur.
- **w** :
 - `-h` : Supprime l'en-tête de la sortie.
- **last** :
 - `-n N` : Affiche les N dernières connexions.
 - `-R` : Supprime l'affichage des adresses IP.

Exemples Pratiques

Vérifier les utilisateurs actifs :

```
who
```

Voir les activités des utilisateurs :

```
w
```

Vérifier l'historique des connexions :

```
last
```

Afficher les 5 dernières connexions :

```
last -n 5
```

Ces commandes sont essentielles pour surveiller les connexions et les utilisateurs actifs sous Linux. Utilisez-les régulièrement pour assurer la sécurité et la gestion efficace de votre système.

V. Automatisation des tâches d'administration avec des scripts Shell.

L'automatisation des tâches d'administration système avec des scripts Shell est un moyen efficace de gagner du temps et de réduire les erreurs humaines. La création et de l'utilisation de scripts Shell pour automatiser des tâches sous Linux.

1. Qu'est-ce qu'un script Shell ?

Un script Shell est un fichier texte contenant une série de commandes Shell qui peuvent être exécutées comme un programme. Ces scripts permettent d'automatiser des tâches répétitives.

2. Création d'un script Shell

a. Écrire un script

Créer un fichier :

```
nano mon_script.sh
```

Ajouter le shebang au début du fichier :

```
#!/bin/bash
```

Cela indique que le script doit être exécuté avec Bash.

Ajouter des commandes :

```
#!/bin/bash
echo "Mise à jour du système..."
sudo apt update && sudo apt upgrade -y
echo "Mise à jour terminée."
```

b. Rendre le script exécutable

```
chmod +x mon_script.sh
```

3. Exécution d'un script

Pour exécuter le script, utilisez la commande suivante :

```
./mon_script.sh
```

4. Utilisation de variables

Vous pouvez utiliser des variables pour rendre votre script plus dynamique.

```
#!/bin/bash
NOM_UTILISATEUR="alice"
echo "Vérification de l'espace disque pour $NOM_UTILISATEUR..."
du -sh /home/$NOM_UTILISATEUR
```

5. Structures de contrôle

Les scripts peuvent inclure des structures de contrôle comme des boucles et des conditions.

a. Conditions

```
#!/bin/bash
if [ -f "/etc/passwd" ]; then
    echo "Le fichier /etc/passwd existe."
else
    echo "Le fichier /etc/passwd n'existe pas."
fi
```

b. Boucles

```
#!/bin/bash
for i in {1..5}; do
    echo "Exécution numéro $i"
done
```

6. Planification des scripts

Pour exécuter des scripts automatiquement à des horaires spécifiques, utilisez `cron`.

a. Éditer la crontab

```
crontab -e
```

b. Ajouter une tâche cron

Pour exécuter le script tous les jours à 2h00 :

```
0 2 * * * /chemin/vers/mon_script.sh
```

7. Exemples de scripts pratiques

a. Sauvegarde de fichiers :

```
#!/bin/bash
tar -czvf sauvegarde_$(date +%F).tar.gz /chemin/du/répertoire
```

b. Nettoyage des fichiers temporaires :

```
#!/bin/bash
rm -rf /tmp/*
```

c. Surveillance des ressources :

```
3. #!/bin/bash
4. echo "Utilisation de l'espace disque :"
5. df -h
6. echo "Utilisation de la mémoire :"
7. free -h
```

Les scripts Shell sont un outil puissant pour automatiser les tâches d'administration sous Linux. En utilisant des scripts, vous pouvez gagner du temps, réduire les erreurs et gérer plus efficacement

votre système. Utiliser la syntaxe des scripts et les commandes Shell pour tirer le meilleur parti de cette fonctionnalité.

Chapitre 4 : Gestion des processus et des services (4H)

I. Gestion des processus (ps, top, kill, nice).

1. Compréhension des processus

Qu'est-ce qu'un processus ?

Un processus est une instance d'un programme en cours d'exécution. Chaque processus possède un identifiant unique (PID) et peut avoir plusieurs.

La gestion des processus est cruciale pour maintenir la performance et la stabilité d'un système Linux. Les commandes clés pour gérer les processus : `ps`, `top`, `kill`, et `nice`.

2. ps

- **Description** : Affiche les processus en cours d'exécution sur le système.
- **Utilisation** :

```
ps [options]
```

Exemples :

- **Afficher tous les processus** :

```
ps aux
```

- `a` : Affiche les processus d'autres utilisateurs.
- `u` : Affiche des informations détaillées sur les processus.
- `x` : Affiche les processus sans terminal de contrôle.

Afficher les processus d'un utilisateur spécifique :

```
ps -u nom_utilisateur
```

3. top

- **Description** : Affiche en temps réel les processus en cours d'exécution, leur utilisation CPU et mémoire.
- **Utilisation** :

```
top
```

- **Fonctionnalités** :
 - **Trier par utilisation CPU ou mémoire** en appuyant sur `M` (mémoire) ou `P` (CPU).
 - **Quitter** en appuyant sur `q`.
- **Options courantes** :
 - `-u nom_utilisateur` : Affiche les processus d'un utilisateur spécifique.

4. kill

- **Description** : Envoie un signal à un processus, souvent utilisé pour terminer un processus.
- **Utilisation** :

```
kill [options] PID
```

Exemples :

- **Terminer un processus** :

```
kill 1234
```

Terminer un processus de manière forcée :

```
kill -9 1234
```

- -9 : Envoie le signal SIGKILL, qui ne peut pas être ignoré.

5. nice et renice

- **Description** : `nice` permet de démarrer un processus avec une priorité spécifiée, tandis que `renice` modifie la priorité d'un processus en cours.
- **Utilisation de `nice`** :

```
nice -n 10 commande
```

- **Exemple** : Démarrer un programme avec une priorité plus basse :

```
nice -n 10 ./mon_programme
```

Utilisation de `renice` :

```
renice -n 5 -p PID
```

Exemple : Modifier la priorité d'un processus existant :

```
renice -n 5 -p 1234
```

6. Exemples pratiques

Lister tous les processus :

```
ps aux
```

Surveiller les processus en temps réel :

```
top
```

Terminer un processus :

```
kill 1234
```

Démarrer un processus avec une priorité spécifique :

```
nice -n 15 ./mon_script.sh
```

Modifier la priorité d'un processus en cours :

```
renice -n 10 -p 1234
```

La gestion des processus est essentielle pour assurer le bon fonctionnement d'un système Linux. Utilisez ces commandes pour surveiller, contrôler et ajuster les processus en cours d'exécution afin d'optimiser les performances de votre système.

II. Gestion des services avec systemd (start, stop, enable).

La gestion des services sous Linux, en particulier avec `systemd`, est essentielle pour contrôler l'exécution des applications et des services. Les commandes clés pour gérer les services avec `systemd`.

1. Qu'est-ce que systemd ?

`systemd` est un système d'initialisation et un gestionnaire de services qui remplace les anciens systèmes comme `init`. Il gère le démarrage des services au démarrage du système, leur arrêt, et leur surveillance.

2. Commandes de base pour gérer les services

a. systemctl

La commande principale pour interagir avec `systemd` est `systemctl`. Voici quelques-unes des utilisations les plus courantes.

b. Démarrer un service

- **Description** : Démarre un service immédiatement.
- **Syntaxe** :

```
sudo systemctl start nom_du_service
```

Exemple :

```
sudo systemctl start apache2
```

c. Arrêter un service

- **Description** : Arrête un service en cours d'exécution.
- **Syntaxe** :

```
sudo systemctl stop nom_du_service
```

Exemple :

```
sudo systemctl stop apache2
```

d. Activer un service

- **Description** : Configure un service pour qu'il démarre automatiquement au démarrage du système.
- **Syntaxe** :

```
sudo systemctl enable nom_du_service
```

Exemple :

```
sudo systemctl enable apache2
```

e. Désactiver un service

- **Description** : Empêche un service de démarrer automatiquement au démarrage du système.
- **Syntaxe** :

```
sudo systemctl disable nom_du_service
```

Exemple :

```
sudo systemctl disable apache2
```

3. Vérification de l'état des services

- **Description** : Affiche l'état actuel d'un service.
- **Syntaxe** :

```
systemctl status nom_du_service
```

Exemple :

```
systemctl status apache2
```

- Cela montrera si le service est actif, inactif, ou en échec, ainsi que des journaux récents.

4. Journalisation avec journalctl

systemd utilise journalctl pour gérer les journaux des services.

- **Afficher les journaux d'un service :**

```
journalctl -u nom_du_service
```

Exemple :

```
journalctl -u apache2
```

5. Exemples pratiques

- Démarrer un service :

```
sudo systemctl start ssh
```

- Arrêter un service :

```
sudo systemctl stop ssh
```

- Activer un service au démarrage :

```
sudo systemctl enable ssh
```

d. **Désactiver un service au démarrage :**

```
sudo systemctl disable ssh
```

e. **Vérifier l'état d'un service :**

```
systemctl status ssh
```

La gestion des services avec `systemd` est un aspect clé de l'administration système sous Linux. Utilisez ces commandes pour démarrer, arrêter, activer ou désactiver des services, ainsi que pour vérifier leur état et consulter leurs journaux. Cela vous aidera à maintenir un environnement stable et performant.

III. Surveillance des performances système (htop, iostat).

La surveillance des performances système est essentielle pour identifier les goulots d'étranglement et assurer un fonctionnement optimal. Deux outils populaires pour la surveillance des performances linux: `htop` et `iostat`.

1. htop

`htop` est un outil interactif de surveillance des processus qui fournit une vue d'ensemble dynamique de l'utilisation des ressources système.

a. Installation

- Sur les systèmes basés sur Debian/Ubuntu :

```
sudo apt install htop
```

Sur les systèmes basés sur Red Hat/Fedora :

```
sudo dnf install htop
```

b. Utilisation

- **Lancer htop :**

```
htop
```

- **Fonctionnalités :**

- Affiche les processus en cours d'exécution, leur utilisation CPU et mémoire.
- Interface colorée et interactive.
- Possibilité de trier les processus par différentes colonnes (CPU, mémoire, etc.) en utilisant les touches de direction.
- **Tuer un processus :** Sélectionnez le processus et appuyez sur `F9`, puis choisissez le signal à envoyer (par exemple, `SIGTERM` ou `SIGKILL`).

c. Commandes utiles dans htop

- **F2** : Accéder au menu de configuration.
- **F3** : Rechercher un processus.
- **F5** : Afficher les processus sous forme d'arbre.
- **F10** : Quitter `htop`.

2. iostat

`iostat` est un outil qui fournit des statistiques sur l'utilisation du CPU et des entrées/sorties des disques.

a. Installation

- Sur les systèmes basés sur Debian/Ubuntu :

```
sudo apt install sysstat
```

Sur les systèmes basés sur Red Hat/Fedora :

```
sudo dnf install sysstat
```

b. Utilisation

- **Afficher les statistiques d'entrée/sortie :**

```
iostat
```

Options utiles :

- **Affichage régulier :**

```
iostat -x 1
```

Cela affichera les statistiques d'entrée/sortie de manière répétée toutes les secondes.

Inclure les statistiques CPU :

```
iostat -c 1
```

c. Interprétation des résultats

- **%iowait** : Pourcentage du temps d'attente d'E/S. Une valeur élevée peut indiquer un goulet d'étranglement.
- **tps** : Transactions par seconde (nombre de lectures et écritures).
- **kB_read/s et kB_wrtn/s** : Kilo-octets lus et écrits par seconde.

3. Exemples pratiques

a. Lancer `htop` pour surveiller les processus :

```
htop
```

Obtenir des statistiques d'E/S avec `iostat` :

```
iostat
```

Afficher les statistiques d'E/S toutes les secondes :

```
iostat -x 1
```

La surveillance des performances système avec des outils comme `htop` et `iostat` permet d'identifier les problèmes potentiels et d'optimiser l'utilisation des ressources. En utilisant ces outils, vous pouvez obtenir une vue d'ensemble de l'état de votre système et prendre des décisions éclairées pour améliorer ses performances.

IV. Planification des tâches avec `cron` et `at`.

La planification des tâches sous Linux est essentielle pour automatiser les opérations répétitives. Deux des outils les plus couramment utilisés pour cela sont `cron` et `at`. Voici le fonctionnement et de leur utilisation.

1. `cron`

`cron` est un démon qui exécute des tâches planifiées à des intervalles réguliers.

a. Crontab

Le fichier `crontab` (tableau de planification) spécifie les tâches à exécuter et leur fréquence. Chaque utilisateur peut avoir son propre fichier `crontab`.

b. Éditer le crontab

Pour éditer votre crontab, utilisez la commande :

```
crontab -e
```

c. Syntaxe d'une entrée crontab

Une entrée dans le crontab suit la syntaxe suivante :

```
* * * * * commande à exécuter
```

• Champs :

- * Minute (0-59)
- * Heure (0-23)
- * Jour du mois (1-31)
- * Mois (1-12)
- * Jour de la semaine (0-7) (0 et 7 représentent dimanche)

d. Exemples

- Exécuter un script tous les jours à 2h00 :

```
0 2 * * * /chemin/vers/mon_script.sh
```

Exécuter une tâche chaque lundi à 14h30 :

```
30 14 * * 1 /chemin/vers/ma_tâche.sh
```

Exécuter un script toutes les heures :

```
0 * * * * /chemin/vers/mon_script.sh
```

e. Commandes utiles

- **Lister les tâches cron actuelles :**

```
crontab -l
```

Supprimer le crontab :

```
crontab -r
```

2. at

`at` est utilisé pour exécuter une tâche une seule fois à un moment spécifié.

a. Installation

Assurez-vous que le service `atd` est en cours d'exécution. Pour l'installer sur Debian/Ubuntu :

```
sudo apt install at
```

b. Utilisation

Pour planifier une tâche avec `at`, utilisez la commande :

```
at [heure] [date]
```

c. Exemples

- **Exécuter une tâche à 15h00 aujourd'hui :**

```
at 15:00
```

Vous pouvez ensuite entrer la commande à exécuter et appuyer sur `Ctrl + D` pour soumettre.

Exécuter une tâche demain à 9h00 :

```
at 09:00 tomorrow
```

Exécuter une tâche à une date précise :

```
at 14:00 2025-02-12
```

d. Vérification des tâches programmées

Pour lister les tâches en attente dans `at`, utilisez :

```
atq
```

e. Annuler une tâche

Pour annuler une tâche planifiée, utilisez :

```
atrm [numéro de tâche]
```

Vous pouvez obtenir le numéro de tâche à partir de la commande `atq`.

La planification des tâches avec `cron` et `at` est une méthode efficace pour automatiser des tâches récurrentes ou uniques sous Linux. En maîtrisant ces outils, vous pouvez améliorer l'efficacité de l'administration système et réduire la charge de travail manuelle.

V. Dépannage des services défaillants et gestion des journaux système.

Le dépannage des services défaillants et la gestion des journaux système sont essentiels pour maintenir la stabilité et la performance d'un système Linux. Voici un guide sur la manière de gérer ces aspects.

1. Dépannage des services défaillants

a. Vérification de l'état d'un service

Utilisez la commande `systemctl` pour vérifier l'état d'un service :

```
systemctl status nom_du_service
```

Cela fournira des informations sur le statut, le PID, et des messages d'erreur récents.

b. Redémarrage d'un service

Si un service échoue, essayez de le redémarrer :

```
sudo systemctl restart nom_du_service
```

c. Vérification des dépendances

Certains services dépendent d'autres. Vérifiez les dépendances avec :

```
systemctl list-dependencies nom_du_service
```

d. Analyser les journaux

Souvent, les journaux contiennent des informations essentielles sur les échecs de services. Utilisez `journalctl` pour afficher les journaux associés à un service :

```
journalctl -u nom_du_service
```

- Options utiles :

- `--since "YYYY-MM-DD HH:MM:SS"` : Pour filtrer les journaux à partir d'une date/heure spécifique.
- `--follow` : Pour suivre les journaux en temps réel.

e. Vérification des fichiers de configuration

Les erreurs dans les fichiers de configuration peuvent provoquer des échecs de service. Vérifiez les fichiers de configuration pour des erreurs de syntaxe ou des valeurs incorrectes.

f. Exécution des diagnostics

Certains services ont des outils de diagnostic intégrés. Consultez la documentation du service pour plus d'informations.

2. Gestion des journaux système

a. Utilisation de `journalctl`

`journalctl` est l'outil principal pour interagir avec les journaux gérés par `systemd`.

b. Afficher les journaux système

Pour afficher tous les journaux :

```
journalctl
```

c. Filtrage des journaux

- Afficher les journaux d'un service spécifique :

```
journalctl -u nom_du_service
```

Afficher les journaux depuis le dernier démarrage :

```
journalctl -b
```

Afficher les erreurs :

```
journalctl -p err
```

d. Options avancées

- Suivre les journaux en temps réel :

```
journalctl -f
```

Limiter le nombre de lignes affichées :

```
journalctl -n 100
```

e. Rotation et conservation des journaux

`systemd` gère la rotation des journaux. Vous pouvez configurer la taille et la durée de conservation des journaux en modifiant `/etc/systemd/journald.conf`.

f. Nettoyage des journaux

Pour nettoyer les journaux anciens, utilisez :

```
sudo journalctl --vacuum-time=7d
```

Cela supprimera les journaux plus anciens que 7 jours.

Le dépannage des services défaillants et la gestion des journaux système sont des compétences essentielles pour tout administrateur Linux. En utilisant les outils appropriés comme `systemctl` et `journalctl`, vous pouvez identifier rapidement les problèmes, analyser les causes sous-jacentes et maintenir un système stable et performant.

Chapitre 5 : Système de fichiers et gestion des disques (5H)

I. Types de systèmes de fichiers (ext4, XFS, Btrfs).

Les systèmes de fichiers sont cruciaux pour la gestion et l'organisation des données sur les systèmes Linux. Trois systèmes de fichiers populaires : `ext4`, `XFS`, et `Btrfs`.

1. ext4 (Fourth Extended Filesystem)

a. Description

`ext4` est l'extension du système de fichiers `ext3`, largement utilisé sur les systèmes Linux. Il est conçu pour être robuste et performant.

b. Caractéristiques

- **Taille maximale du système de fichiers** : 1 Exaoctet (EB).
- **Taille maximale d'un fichier** : 16 To.
- **Journaling** : Utilise un journal pour assurer l'intégrité des données, ce qui améliore la récupération après une panne.
- **Allocation dynamique** : Améliore l'utilisation de l'espace disque en permettant la réservation dynamique d'inodes.
- **Performances** : Optimisé pour les performances, avec un système d'allocation des blocs plus efficace.

c. Cas d'utilisation

- Idéal pour les systèmes de fichiers de serveurs et de postes de travail, grâce à sa stabilité et ses performances.

2. XFS

a. Description

- **XFS** est un système de fichiers hautes performances conçu pour gérer de grandes quantités de données et de fichiers.

b. Caractéristiques

- **Taille maximale du système de fichiers** : 8 Exaoctets (EB).
- **Taille maximale d'un fichier** : 8 To.
- **Journaling** : Utilise le journaling pour la récupération après sinistre.
- **Allocation de blocs** : Utilise une allocation de blocs à la volée, ce qui rend XFS particulièrement efficace pour les systèmes avec de nombreux fichiers ou de gros fichiers.
- **Scalabilité** : Très scalable, idéal pour les environnements de serveur.

c. Cas d'utilisation

- Souvent utilisé dans des environnements à forte charge, comme les bases de données et les systèmes de fichiers multimédias.

3. Btrfs (B-tree File System)

a. Description

Btrfs est un système de fichiers moderne qui introduit des fonctionnalités avancées pour la gestion des données.

b. Caractéristiques

- **Snapshots** : Permet de créer des snapshots instantanés du système de fichiers, facilitant la sauvegarde et la restauration.
- **Compression** : Prend en charge la compression des données à la volée.
- **RAID intégré** : Offre des fonctionnalités RAID, permettant de gérer plusieurs disques facilement.
- **Scalabilité** : Conçu pour être scalable, avec une taille maximale de système de fichiers de 16 Exaoctets.
- **Vérification d'intégrité** : Intègre des mécanismes pour vérifier l'intégrité des données.

c. Cas d'utilisation

- Idéal pour des environnements nécessitant des fonctionnalités avancées comme les snapshots, la compression, et la gestion des disques.

Chacun de ces systèmes de fichiers a ses propres forces et est adapté à des cas d'utilisation spécifiques. Le choix du système de fichiers dépend souvent des besoins particuliers de l'environnement, qu'il s'agisse de performances, de gestion de données ou de fonctionnalités avancées.

II. Montage et démontage des systèmes de fichiers (mount, umount).

Le montage et le démontage des systèmes de fichiers sont des opérations fondamentales dans la gestion des disques et des périphériques de stockage sous Linux. Voici un guide sur les commandes `mount` et `umount`.

1. Montages des systèmes de fichiers

a. Commande mount

La commande `mount` est utilisée pour attacher un système de fichiers à un point de montage dans l'arborescence du système.

b. Syntaxe

```
mount [options] périphérique point_de_montage
```

c. Exemples

Monter un système de fichiers :

```
sudo mount /dev/sdb1 /mnt/mon_dossier
```

Ici, `/dev/sdb1` est la partition à monter et `/mnt/mon_dossier` est le point de montage.

Monter un système de fichiers avec des options :

```
sudo mount -o ro /dev/sdb1 /mnt/mon_dossier
```

- `-o ro` : Monte le système de fichiers en mode lecture seule.

Monter un système de fichiers ISO :

```
sudo mount -o loop fichier.iso /mnt/mon_iso
```

- `-o loop` : Utilisé pour monter un fichier image ISO comme un système de fichiers.

d. Afficher les systèmes de fichiers montés

Pour afficher tous les systèmes de fichiers actuellement montés :

```
mount
```

2. Démontage des systèmes de fichiers

a. Commande umount

La commande `umount` est utilisée pour démonter un système de fichiers précédemment monté.

b. Syntaxe

```
umount [options] point_de_montage
```

ou

```
umount [options] périphérique
```

c. Exemples

Démonter un système de fichiers :

```
sudo umount /mnt/mon_dossier
```

Démonter un système de fichiers par périphérique :

```
sudo umount /dev/sdb1
```

Forcer le démontage :

Si un système de fichiers ne peut pas être démonté (par exemple, s'il est occupé), vous pouvez forcer le démontage :

```
sudo umount -l /mnt/mon_dossier
```

- `-l` : Démontage paresseux (lazy), qui libère le point de montage lorsque le système de fichiers n'est plus utilisé.

d. Vérification de l'état de démontage

Après avoir démonté un système de fichiers, vous pouvez vérifier qu'il n'est plus monté en utilisant :

```
mount | grep /mnt/mon_dossier
```

Si aucune sortie n'est retournée, le démontage a réussi.

Le montage et le démontage des systèmes de fichiers sont des opérations essentielles pour gérer les périphériques de stockage sous Linux. En utilisant les commandes `mount` et `umount`, vous pouvez facilement accéder et gérer vos systèmes de fichiers. Assurez-vous de démonter correctement les systèmes de fichiers avant de retirer des périphériques pour éviter la perte de données.

III. Gestion des partitions et des volumes logiques (fdisk, lsblk, LVM).

La gestion des partitions et des volumes logiques est essentielle pour l'organisation et l'optimisation de l'espace de stockage sur un système Linux. Les principaux outils : `fdisk`, `lsblk`, et LVM (Logical Volume Management) sont utilisés à cet effet.

1. fdisk

a. Description

`fdisk` est un utilitaire de partitionnement de disque qui permet de créer, supprimer et modifier des partitions sur des disques durs.

b. Utilisation

Lister les disques et partitions :

```
sudo fdisk -l
```

Modifier un disque :

Pour ouvrir un disque spécifique (par exemple, `/dev/sda`) :

```
sudo fdisk /dev/sda
```

c. Commandes courantes

- **Créer une nouvelle partition :**
 - Tapez `n` pour créer une nouvelle partition.
- **Supprimer une partition :**
 - Tapez `d` pour supprimer une partition.
- **Changer le type de partition :**
 - Tapez `t` et suivez les instructions.
- **Sauvegarder les modifications et quitter :**
 - Tapez `w` pour écrire les changements sur le disque.

2. lsblk

a. Description

`lsblk` est un outil qui affiche les informations sur les périphériques de bloc, y compris les partitions et les volumes.

b. Utilisation

Lister les périphériques et les partitions :

```
lsblk
```

Afficher des informations détaillées :

```
lsblk -f
```

c. Options utiles

Afficher l'espace utilisé et disponible :

```
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
```

3. LVM (Logical Volume Management)

a. Description

LVM permet de gérer des volumes logiques, offrant plus de flexibilité que le partitionnement traditionnel. Il permet de redimensionner, créer et supprimer des volumes facilement.

b. Concepts clés

- **Physical Volume (PV) :** Disques ou partitions utilisés par LVM.
- **Volume Group (VG) :** Un groupe de volumes physiques.
- **Logical Volume (LV) :** Volumes créés à l'intérieur d'un groupe de volumes.

c. Commandes LVM

- Créer un volume physique :

```
sudo pvcreate /dev/sdb1
```

Créer un groupe de volumes :

```
sudo vgcreate mon_groupe /dev/sdb1
```

Créer un volume logique :

```
sudo lvcreate -n mon_volume -L 10G mon_groupe
```

Monter un volume logique :

```
sudo mount /dev/mon_groupe/mon_volume /mnt
```

Afficher l'état des volumes :

```
sudo lvdisplay
```

La gestion des partitions et des volumes logiques est cruciale pour une utilisation efficace de l'espace de stockage. Utilisez `fdisk` pour le partitionnement, `lsblk` pour l'affichage des informations sur les disques, et LVM pour une gestion flexible des volumes logiques.

IV. Analyse et gestion de l'espace disque (df, du, ncd).

L'analyse et la gestion de l'espace disque sont essentielles pour maintenir un système Linux performant. Les outils couramment utilisés : `df`, `du`, et `ncdu`.

1. df (Disk Free)

a. Description

`df` est une commande qui affiche des informations sur l'espace disque utilisé et disponible sur les systèmes de fichiers.

b. Utilisation

Afficher l'espace disque :

```
df
```

Afficher l'espace en format lisible (Go, To) :

```
df -h
```

Afficher les systèmes de fichiers avec des informations spécifiques :

```
df -T
```

c. Options utiles

Afficher uniquement les systèmes de fichiers montés :

```
df -x tmpfs
```

2. du (Disk Usage)

a. Description

`du` est utilisé pour estimer l'espace utilisé par des fichiers et des répertoires.

b. Utilisation

Afficher l'espace utilisé par un répertoire :

```
du /chemin/vers/repertoire
```

Afficher l'espace en format lisible :

```
du -h /chemin/vers/repertoire
```

Afficher le total pour un répertoire :

```
du -sh /chemin/vers/repertoire
```

c. Options utiles

Afficher l'espace utilisé par chaque sous-répertoire :

```
du -h --max-depth=1 /chemin/vers/repertoire
```

3. ncd (NCurses Disk Usage)

a. Description

`ncdu` est un outil d'analyse de l'espace disque basé sur l'interface en ligne de commande qui offre une interface interactive pour explorer l'utilisation du disque.

b. Installation

Pour installer `ncdu` sur Debian/Ubuntu :

```
sudo apt install ncd
```

c. Utilisation

Analyser un répertoire :

```
ncdu /chemin/vers/repertoire
```


d. Fonctionnalités

- Interface interactive pour naviguer dans les répertoires.
- Permet de supprimer des fichiers et des répertoires directement dans l'interface.

Ces outils permettent de surveiller et de gérer efficacement l'espace disque. Utilisez `df` pour une vue d'ensemble de l'espace disponible, `du` pour une analyse détaillée de l'utilisation des fichiers et répertoires, et `ncdu` pour une interface interactive et conviviale.

V. Sauvegarde et restauration des données avec `rsync` et `tar`.

La sauvegarde et la restauration des données sont des tâches essentielles pour la protection des informations sur un système Linux. Voici deux outils populaires : `rsync` et `tar`.

1. `rsync`

a. Description

`rsync` est un outil de synchronisation de fichiers qui permet de transférer et de sauvegarder des fichiers de manière efficace en ne copiant que les différences entre les fichiers source et destination.

b. Utilisation

Sauvegarder un répertoire localement :

```
rsync -av /source/ /destination/
```

- `-a` : Mode archive, préserve les permissions, les liens, etc.
- `-v` : Affiche les détails des fichiers en cours de transfert.

Sauvegarder vers un serveur distant :

```
rsync -av /source/ user@remote_host:/destination/
```

Synchroniser un répertoire à partir d'un serveur distant :

```
rsync -av user@remote_host:/source/ /destination/
```

c. Options utiles

- **Supprimer les fichiers dans la destination qui ne sont plus dans la source :**

```
rsync -av --delete /source/ /destination/
```

Exclure des fichiers ou répertoires spécifiques :

```
rsync -av --exclude 'fichier_ou_repertoire' /source/ /destination/
```

2. tar

a. Description

`tar` est un outil de création d'archives qui permet de regrouper plusieurs fichiers et répertoires en un seul fichier, souvent utilisé pour la sauvegarde.

b. Utilisation

Créer une archive tar :

```
tar -cvf archive.tar /chemin/vers/dossier
```

- `-c` : Crée une nouvelle archive.
- `-v` : Affiche les fichiers ajoutés à l'archive.
- `-f` : Spécifie le nom du fichier d'archive.

Créer une archive compressée (gzip) :

```
tar -czvf archive.tar.gz /chemin/vers/dossier
```

Extraire une archive :

```
tar -xvf archive.tar
```

Extraire une archive compressée :

```
tar -xzvf archive.tar.gz
```

c. Options utiles

Lister le contenu d'une archive sans l'extraire :

```
tar -tvf archive.tar
```

Ajouter des fichiers à une archive existante :

```
tar -rvf archive.tar nouveau_fichier
```

`rsync` et `tar` sont des outils puissants pour la sauvegarde et la restauration des données sur Linux. Utilisez `rsync` pour des sauvegardes incrémentielles et une synchronisation efficace, tandis que `tar` est idéal pour créer des archives de fichiers. Choisissez l'outil en fonction de vos besoins spécifiques de sauvegarde.

Chapitre 6 : Réseau sous Linux (4H)

I. Configuration des interfaces réseau (ifconfig, ip, nmcli).

La configuration des interfaces réseau est essentielle pour établir des connexions réseau sur un système Linux. Voici un aperçu des outils principaux : `ifconfig`, `ip`, et `nmcli`.

1. ifconfig

a. Description

`ifconfig` est un utilitaire de gestion des interfaces réseau qui permet de configurer, afficher et désactiver les interfaces.

b. Utilisation

Afficher la configuration actuelle des interfaces :

```
ifconfig
```

Configurer une adresse IP :

```
sudo ifconfig eth0 192.168.1.100 netmask 255.255.255.0
```

Activer une interface :

```
sudo ifconfig eth0 up
```

Désactiver une interface :

```
sudo ifconfig eth0 down
```

c. Remarque

`ifconfig` est obsolète sur certaines distributions modernes au profit de la commande `ip`.

2. ip

a. Description

La commande `ip` est un outil puissant et moderne pour gérer les interfaces réseau, les routes et d'autres paramètres réseau.

b. Utilisation

Afficher les interfaces réseau :

```
ip addr show
```

Configurer une adresse IP :

```
sudo ip addr add 192.168.1.100/24 dev eth0
```

Activer une interface :

```
sudo ip link set eth0 up
```

Désactiver une interface :

```
sudo ip link set eth0 down
```

Afficher les routes IP :

```
ip route show
```

c. Avantages

`ip` offre plus de fonctionnalités et est recommandé pour une utilisation moderne.

3. nmcli

a. Description

`nmcli` est un outil en ligne de commande pour gérer NetworkManager, qui gère les connexions réseau sur les systèmes basés sur Linux.

b. Utilisation

Afficher les connexions réseau :

```
nmcli connection show
```

Configurer une nouvelle connexion :

```
nmcli connection add type ethernet ifname eth0 con-name ma_connexion ip4 192.168.1.100/24
```

Activer une connexion :

```
nmcli connection up ma_connexion
```

Désactiver une connexion :

```
nmcli connection down ma_connexion
```

c. Remarque

`nmcli` est particulièrement utile pour gérer des connexions réseau sur des systèmes ayant un environnement graphique.

Pour la configuration des interfaces réseau sur Linux, utilisez `ifconfig` pour des tâches simples, `ip` pour une gestion moderne et complète, et `nmcli` pour une intégration avec NetworkManager. Le choix de l'outil dépend de vos besoins spécifiques et de la configuration de votre système.

II. Paramétrage des services réseau (SSH, FTP, DHCP).

Le paramétrage des services réseau est essentiel pour assurer la connectivité et la gestion des systèmes sur un réseau. Voici un aperçu des services courants : SSH, FTP et DHCP.

1. SSH (Secure Shell)

a. Description

SSH est un protocole permettant d'accéder de manière sécurisée à un système distant. Il chiffre les communications pour garantir la confidentialité.

b. Installation

Pour installer le serveur SSH (OpenSSH) sur une distribution basée sur Debian/Ubuntu :

```
sudo apt update
sudo apt install openssh-server
```

c. Configuration

- **Fichier de configuration** : `/etc/ssh/sshd_config`
- **Modifier les paramètres** (par exemple, changer le port par défaut) :

```
sudo nano /etc/ssh/sshd_config
```

Redémarrer le service pour appliquer les modifications :

```
sudo systemctl restart ssh
```

d. Connexion

Pour se connecter à un serveur SSH :

```
ssh user@adresse_ip
```

2. FTP (File Transfer Protocol)

a. Description

FTP est un protocole utilisé pour transférer des fichiers entre un client et un serveur.

b. Installation

Pour installer un serveur FTP (vsftpd) sur Debian/Ubuntu :

```
sudo apt update
sudo apt install vsftpd
```

c. Configuration

Fichier de configuration : `/etc/vsftpd.conf`

Modifier les paramètres (par exemple, activer les connexions anonymes ou restreindre les utilisateurs) :

```
sudo nano /etc/vsftpd.conf
```

Redémarrer le service pour appliquer les modifications :

```
sudo systemctl restart vsftpd
```

d. Connexion

Pour se connecter à un serveur FTP :

```
ftp adresse_ip
```

III. DHCP (Dynamic Host Configuration Protocol)

1. Description

DHCP est un protocole qui permet d'attribuer automatiquement des adresses IP et d'autres paramètres réseau à des dispositifs sur un réseau.

2. Installation

Pour installer un serveur DHCP (isc-dhcp-server) sur Debian/Ubuntu :

```
sudo apt update
sudo apt install isc-dhcp-server
```

3. Configuration

Fichier de configuration : `/etc/dhcp/dhcpd.conf`

Définir un sous-réseau (exemple) :

```
• subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200;
    option routers 192.168.1.1;
    option domain-name-servers 8.8.8.8;
}
```

Redémarrer le service pour appliquer les modifications :

```
sudo systemctl restart isc-dhcp-server
```

4. Vérification

Pour vérifier si le service DHCP fonctionne :

```
sudo systemctl status isc-dhcp-server
```

Le paramétrage des services réseau tels que SSH, FTP et DHCP est crucial pour la gestion et la connectivité dans un environnement réseau. Assurez-vous de sécuriser correctement ces services et de configurer les fichiers de manière appropriée pour répondre à vos besoins.

IV. Outils de diagnostic réseau (ping, netstat, traceroute, nmap).

Les outils de diagnostic réseau sont essentiels pour analyser et résoudre les problèmes de connectivité sur un réseau. Voici un aperçu de quelques outils courants : ping, netstat, traceroute, et nmap.

1. ping

a. Description

`ping` est un outil qui teste la connectivité entre votre machine et une autre machine sur le réseau en envoyant des paquets ICMP Echo Request.

b. Utilisation

Tester la connectivité :

```
ping adresse_ip_ou_nom_domaine
```

Envoyer un nombre spécifique de paquets :

```
ping -c 4 adresse_ip_ou_nom_domaine
```

c. Analyse des résultats

- **Temps de réponse** : Indique le temps qu'il a fallu pour que le paquet aille et revienne.
- **Perte de paquets** : Si des paquets sont perdus, cela peut indiquer un problème de connectivité.

2. netstat

a. Description

`netstat` est un outil qui affiche les connexions réseau, les tables de routage et d'autres statistiques réseau.

b. Utilisation

Afficher les connexions actives :

```
netstat -a
```

Afficher les connexions avec les processus associés :

```
netstat -tulnp
```

Afficher les statistiques réseau :

```
netstat -s
```

c. Remarque

`netstat` est souvent remplacé par `ss` dans les systèmes modernes pour une gestion plus avancée des connexions.

3. traceroute

a. Description

`tracert` est un outil qui trace le chemin qu'un paquet prend pour atteindre une destination, en affichant chaque routeur traversé.

b. Utilisation

- Tracer une route vers une adresse :

```
tracert adresse_ip_ou_nom_domaine
```

c. Analyse des résultats

Temps de réponse à chaque saut : Indique combien de temps chaque routeur prend pour répondre.

Identification des points de défaillance : Si un saut ne répond pas, cela peut indiquer un problème à ce niveau.

4. nmap

a. Description

`nmap` (Network Mapper) est un outil de scan de réseau qui permet d'explorer les réseaux et de découvrir les hôtes et services.

b. Installation

Pour installer `nmap` sur Debian/Ubuntu :

```
sudo apt update  
sudo apt install nmap
```

c. Utilisation

Scanner un hôte pour découvrir les ports ouverts :

```
nmap adresse_ip
```

Scanner une plage d'adresses IP :

```
nmap 192.168.1.1-254
```

Scanner pour déterminer le système d'exploitation :

```
nmap -O adresse_ip
```

d. Remarque

`nmap` doit être utilisé avec précaution et dans le respect des lois et règlements, car le scan de réseaux sans autorisation peut être illégal.

Ces outils de diagnostic réseau sont essentiels pour analyser et résoudre les problèmes de connectivité. Utilisez `ping` pour vérifier la connectivité, `netstat` pour afficher les connexions

actives, `traceroute` pour diagnostiquer le chemin des paquets, et `nmap` pour explorer les hôtes et les services sur le réseau.

V. Sécurisation des connexions réseau avec les pare-feux (iptables, UFW).

La sécurisation des connexions réseau est cruciale pour protéger votre système Linux contre les accès non autorisés et les attaques. Voici un aperçu de deux outils couramment utilisés pour gérer les pare-feux : `iptables` et `UFW` (Uncomplicated Firewall).

1. iptables

a. Description

`iptables` est un outil de filtrage de paquets qui permet de configurer des règles de pare-feu pour contrôler le trafic réseau entrant et sortant.

b. Utilisation de base

Afficher les règles actuelles :

```
sudo iptables -L
```

Ajouter une règle pour autoriser le trafic HTTP (port 80) :

```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

Bloquer une adresse IP spécifique :

```
sudo iptables -A INPUT -s 192.168.1.100 -j DROP
```

Enregistrer les règles :

Pour sauvegarder les règles afin qu'elles persistent après un redémarrage, utilisez :

```
sudo iptables-save > /etc/iptables/rules.v4
```

c. Remarque

`iptables` nécessite une bonne compréhension des règles de filtrage, car une mauvaise configuration peut bloquer l'accès légitime au réseau.

2. UFW (Uncomplicated Firewall)

a. Description

`UFW` est une interface simplifiée pour `iptables` qui facilite la gestion des règles de pare-feu. Il est conçu pour être facile à utiliser, même pour les utilisateurs novices.

b. Installation

Sur Debian/Ubuntu, vous pouvez installer `UFW` avec :

```
sudo apt install ufw
```

c. Utilisation de base

Activer UFW :

```
sudo ufw enable
```

Afficher le statut :

```
sudo ufw status
```

Autoriser le trafic HTTP :

```
sudo ufw allow http
```

Bloquer une adresse IP :

```
sudo ufw deny from 192.168.1.100
```

Désactiver UFW :

```
sudo ufw disable
```

d. Avantages

UFW est plus accessible pour les nouveaux utilisateurs qui souhaitent configurer rapidement un pare-feu sans plonger dans les détails complexes d'*iptables*.

Pour sécuriser vos connexions réseau sur Linux, utilisez *iptables* pour une configuration avancée et personnalisée des règles de pare-feu, ou UFW pour une gestion simplifiée et rapide. Choisissez l'outil qui correspond le mieux à votre niveau de compétence et à vos besoins en matière de sécurité.

VI. Configuration des serveurs DNS et DHCP sous Linux.

Configuration d'un serveur DNS

1. Description

BIND (Berkeley Internet Name Domain) est un serveur DNS largement utilisé sur les systèmes Linux.

2. Installation

Pour installer BIND sur Debian/Ubuntu, exécutez :

```
sudo apt update  
sudo apt install bind9
```

3. Configuration

a. Fichier principal de configuration

Le fichier principal se trouve à `/etc/bind/named.conf`. Ajoutez une zone DNS en insérant la ligne suivante :

```
zone "exemple.com" {
    type master;
    file "/etc/bind/db.exemple.com";
};
```

b. Créer un fichier de zone

Créez un fichier de zone à `/etc/bind/db.exemple.com` :

```
$TTL      604800
@         IN      SOA      ns.exemple.com. admin.exemple.com. (
                        1      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        604800 ) ; Negative Cache TTL
;
@         IN      NS       ns.exemple.com.
ns        IN      A        192.168.1.10
@         IN      A        192.168.1.20
www       IN      A        192.168.1.20
```

c. Redémarrer le service BIND

Pour appliquer les modifications, redémarrez le service :

```
sudo systemctl restart bind9
```

4. Vérification

Pour vérifier la configuration du DNS, utilisez :

```
dig @localhost exemple.com
```

Configuration d'un serveur DHCP

1. Description

Le serveur DHCP (Dynamic Host Configuration Protocol) attribue automatiquement des adresses IP et d'autres paramètres réseau.

2. Installation

Pour installer un serveur DHCP sur Debian/Ubuntu, exécutez :

```
sudo apt update
sudo apt install isc-dhcp-server
```

3. Configuration

a. Fichier principal de configuration

Le fichier principal se trouve à `/etc/dhcp/dhcpd.conf`. Voici un exemple de configuration :

```
subnet 192.168.1.0 netmask 255.255.255.0 {
    range 192.168.1.100 192.168.1.200; # Plage d'adresses disponibles
    option routers 192.168.1.1; # Passerelle par défaut
    option domain-name-servers 8.8.8.8, 8.8.4.4; # Serveurs DNS
    option domain-name "exemple.com"; # Nom de domaine
}
```

b. Spécifier l'interface réseau

Modifiez `/etc/default/isc-dhcp-server` pour spécifier l'interface réseau :

```
INTERFACESv4="eth0"
```

4. Redémarrer le service DHCP

Redémarrez le service pour appliquer les modifications :

```
sudo systemctl restart isc-dhcp-server
```

5. Vérification

Pour vérifier que le serveur DHCP fonctionne, utilisez :

```
sudo systemctl status isc-dhcp-server
```

Pour voir les journaux :

```
sudo tail -f /var/log/syslog
```

La configuration des serveurs DNS (avec BIND) et DHCP est essentielle pour la gestion d'un réseau. BIND résout les noms de domaine, tandis que le serveur DHCP attribue des adresses IP automatiquement. Testez et ajustez les configurations selon vos besoins pour garantir un fonctionnement optimal.

Chapitre 7 : Sécurité des systèmes Linux (6H)

I. Gestion des droits d'accès et des utilisateurs privilégiés (sudo, polkit).

La gestion des droits d'accès et des utilisateurs privilégiés est essentielle pour la sécurité d'un système Linux. Voici un aperçu des outils couramment utilisés : `sudo` et `polkit`.

1. Gestion des droits d'accès avec sudo

a. Description

`sudo` permet aux utilisateurs d'exécuter des commandes avec les privilèges d'un autre utilisateur, généralement l'utilisateur root. Cela permet de restreindre l'accès aux commandes sensibles tout en offrant une méthode sécurisée pour les exécuter.

b. Configuration de sudo

Le fichier de configuration principal de `sudo` est `/etc/sudoers`. Il est recommandé de modifier ce fichier avec la commande `visudo` pour éviter les erreurs de syntaxe.

c. Exemple de configuration

Pour donner à un utilisateur nommé `utilisateur` le droit d'exécuter toutes les commandes :

```
utilisateur ALL=(ALL:ALL) ALL
```

Pour permettre à `utilisateur` d'exécuter uniquement certaines commandes :

```
utilisateur ALL=(ALL:ALL) /usr/bin/apt, /usr/bin/systemctl
```

d. Utilisation de sudo

Pour exécuter une commande avec `sudo`, il suffit de précéder la commande par `sudo` :

```
sudo apt update
```

e. Journalisation

`sudo` journalise toutes les actions dans `/var/log/auth.log`, ce qui permet de suivre les usages et de détecter des comportements suspects.

2. Gestion des droits d'accès avec polkit

a. Description

`polkit` (PolicyKit) est un framework utilisé pour définir et gérer les autorisations des utilisateurs dans un environnement Linux. Il est souvent utilisé pour contrôler l'accès à des fonctionnalités spécifiques des services.

b. Configuration de polkit

Les règles de `polkit` peuvent être définies dans le répertoire `/etc/polkit-1/rules.d/`. Les fichiers de règles doivent avoir une extension `.rules`.

c. Exemple de règle

Pour créer une règle qui permet à l'utilisateur `utilisateur` d'arrêter le service `NetworkManager` sans mot de passe, créez un fichier nommé `/etc/polkit-1/rules.d/50-allow-user.rules` avec le contenu suivant :

javascript

```
polkit.addRule(function(action, subject) {  
    if (action.id == "org.freedesktop.NetworkManager.stop" &&  
subject.isInGroup("utilisateur")) {  
        return polkit.Result.YES;  
    }  
});
```

```
}  
});
```

d. Utilisation

`polkit` est souvent utilisé avec des commandes qui nécessitent des privilèges administratifs, comme `systemctl`. Si un utilisateur essaie d'exécuter une commande protégée, une invite graphique peut apparaître pour demander l'authentification.

La gestion des droits d'accès et des utilisateurs privilégiés avec `sudo` et `polkit` est cruciale pour la sécurité de votre système Linux. `sudo` fournit un moyen contrôlé d'exécuter des commandes avec des privilèges élevés, tandis que `polkit` permet une gestion fine des autorisations pour les actions spécifiques. Assurez-vous de configurer ces outils correctement pour protéger votre environnement.

II. Mise à jour des systèmes et gestion des correctifs de sécurité (apt, yum).

La mise à jour des systèmes et la gestion des correctifs de sécurité sont essentielles pour maintenir la sécurité et la stabilité de votre environnement Linux. Voici un aperçu des outils couramment utilisés : `apt` pour les distributions basées sur Debian et `yum` (ou `dnf`) pour les distributions basées sur Red Hat.

1. Mise à jour des systèmes avec apt

a. Description

`apt` (Advanced Package Tool) est un gestionnaire de paquets pour les distributions Debian et Ubuntu.

b. Commandes de base

Mise à jour des listes de paquets

Avant d'installer des mises à jour, il est conseillé de mettre à jour la liste des paquets disponibles :

```
sudo apt update
```

Mise à jour des paquets installés

Pour mettre à jour tous les paquets installés vers les dernières versions disponibles :

```
sudo apt upgrade
```

Mise à niveau complète du système

Pour effectuer une mise à niveau complète, qui peut inclure la suppression de paquets obsolètes :

```
sudo apt full-upgrade
```

Installation des mises à jour de sécurité

Pour installer uniquement les mises à jour de sécurité, vous pouvez utiliser :

```
sudo apt install unattended-upgrades
```

Et configurer les mises à jour automatiques dans `/etc/apt/apt.conf.d/50unattended-upgrades`.

c. Gestion des mises à jour

Vérifiez régulièrement les mises à jour disponibles et appliquez-les pour assurer la sécurité de votre système.

2. Mise à jour des systèmes avec yum / dnf

a. Description

`yum` (Yellowdog Updater Modified) est un gestionnaire de paquets pour les distributions basées sur Red Hat, comme CentOS et Fedora. `dnf` (Dandified YUM) est son remplaçant plus récent.

b. Commandes de base

Mise à jour des paquets

Pour mettre à jour tous les paquets installés :

```
sudo yum update
```

ou, pour `dnf` :

```
sudo dnf upgrade
```

Installation de mises à jour de sécurité

Pour installer uniquement les mises à jour de sécurité avec `yum` :

```
sudo yum --security update
```

Avec `dnf`, utilisez :

```
sudo dnf update --security
```

c. Gestion des mises à jour

Il est recommandé de vérifier régulièrement les mises à jour disponibles et de les appliquer pour protéger votre système contre les vulnérabilités.

La mise à jour régulière des systèmes et l'application des correctifs de sécurité sont cruciales pour maintenir un environnement Linux sécurisé. Utilisez `apt` pour les distributions basées sur Debian et `yum` ou `dnf` pour celles basées sur Red Hat. Assurez-vous de planifier des mises à jour régulières et de configurer des mises à jour automatiques si nécessaire.

3. Sécurisation des connexions avec SSH et pare-feu (fail2ban, ufw).

La sécurisation des connexions SSH et l'utilisation d'un pare-feu sont essentielles pour protéger un système Linux contre les accès non autorisés. Voici comment mettre en œuvre ces mesures de sécurité à l'aide de SSH, `fail2ban`, et `ufw`.

III. Sécurisation des connexions SSH

1. Configuration de SSH

Installation

La plupart des distributions Linux installent `OpenSSH` par défaut. Pour l'installer manuellement :

```
sudo apt install openssh-server # Pour Debian/Ubuntu
sudo yum install openssh-server # Pour CentOS/RHEL
```

Fichier de configuration

Le fichier de configuration se trouve à `/etc/ssh/sshd_config`. Voici quelques recommandations de sécurité :

a. Changer le port par défaut (optionnel) :

```
Port 2222 # Remplacez 2222 par un port de votre choix
```

Désactiver l'accès root :

```
PermitRootLogin no
```

Utiliser l'authentification par clé publique :

Générez une paire de clés :

```
ssh-keygen
```

Copiez la clé publique sur le serveur :

```
ssh-copy-id user@hostname
```

Désactiver l'authentification par mot de passe (si vous utilisez des clés) :

```
PasswordAuthentication no
```

Redémarrer le service SSH

Après les modifications, redémarrez le service SSH :

```
sudo systemctl restart sshd
```

b. Vérification de la configuration

Testez la connexion SSH pour vous assurer que tout fonctionne correctement.

2. Sécurisation avec fail2ban

a. Description

fail2ban est un outil qui surveille les fichiers journaux pour détecter les tentatives de connexion échouées et bannit les adresses IP suspectes.

b. Installation

Pour installer fail2ban :

```
sudo apt install fail2ban # Pour Debian/Ubuntu
sudo yum install fail2ban # Pour CentOS/RHEL
```

c. Configuration

Le fichier de configuration principal est `/etc/fail2ban/jail.conf`. Créez une copie pour la personnaliser :

```
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

Exemple de configuration

Pour protéger SSH, activez le jail SSH dans `/etc/fail2ban/jail.local` :

ini

```
[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log # Pour Debian/Ubuntu
# logpath = /var/log/secure # Pour CentOS/RHEL
maxretry = 5
bantime = 600 # Durée du ban en secondes
```

d. Démarrer et activer fail2ban

```
sudo systemctl start fail2ban
sudo systemctl enable fail2ban
```

3. Mise en place d'un pare-feu avec ufw

a. Description

ufw (Uncomplicated Firewall) est un outil de gestion de pare-feu simplifié pour Linux.

b. Installation

ufw est généralement préinstallé sur les systèmes basés sur Ubuntu. Sinon, installez-le :

```
sudo apt install ufw # Pour Debian/Ubuntu
```

c. Configuration

Activer `ufw`

```
sudo ufw enable
```

Autoriser les connexions SSH

Assurez-vous que les connexions SSH sont autorisées :

```
sudo ufw allow OpenSSH
# ou, si vous avez changé le port :
sudo ufw allow 2222/tcp # Remplacez 2222 par votre port SSH
```

Bloquer les connexions entrantes non désirées

Par défaut, `ufw` bloque les connexions entrantes. Vous pouvez vérifier l'état :

```
sudo ufw status
```

d. Vérification

Testez votre configuration SSH après l'activation d'`ufw` pour vous assurer que vous pouvez toujours vous connecter.

La sécurisation des connexions SSH et l'utilisation d'un pare-feu comme `ufw`, combinées avec `fail2ban`, renforcent considérablement la sécurité de votre système Linux. Assurez-vous de suivre ces étapes pour protéger vos serveurs contre les accès non autorisés.

4. Chiffrement des données et gestion des clés (GPG, OpenSSL).

Le chiffrement des données et la gestion des clés sont des aspects cruciaux de la sécurité des informations. Voici un aperçu des outils couramment utilisés pour le chiffrement, notamment GPG et OpenSSL.

IV. Chiffrement des données avec GPG

1. Chiffrement des données avec OpenSSL

a. Description

GPG (GNU Privacy Guard) est un outil de chiffrement basé sur la norme OpenPGP. Il permet de chiffrer des données et de signer des communications.

b. Installation

Pour installer GPG sur Debian/Ubuntu :

```
sudo apt install gnupg
```

Sur CentOS/RHEL :

```
sudo yum install gnupg
```

c. Créer une paire de clés

Pour chiffrer des données, vous devez d'abord créer une paire de clés :

```
gpg --full-generate-key
```

Suivez les instructions pour choisir le type de clé, la taille, et définir une phrase de passe.

d. Chiffrer un fichier

Pour chiffrer un fichier, utilisez la commande suivante :

```
gpg -e -r "Nom de l'utilisateur" fichier.txt
```

Cela crée un fichier chiffré nommé `fichier.txt.gpg`.

e. Déchiffrer un fichier

Pour déchiffrer le fichier, utilisez :

```
gpg -d fichier.txt.gpg > fichier.txt
```

f. Exporter et importer des clés

Pour exporter votre clé publique :

```
gpg --export -a "Nom de l'utilisateur" > public_key.asc
```

Pour importer une clé publique :

```
gpg --import public_key.asc
```

2. Chiffrement des données avec OpenSSL

a. Description

OpenSSL est une bibliothèque robuste pour le chiffrement et la gestion des certificats. Il est utilisé pour plusieurs tâches de sécurité, y compris le chiffrement de fichiers et la gestion des certificats SSL/TLS.

b. Installation

OpenSSL est souvent préinstallé. Pour l'installer manuellement :

```
sudo apt install openssl # Pour Debian/Ubuntu  
sudo yum install openssl # Pour CentOS/RHEL
```

c. Chiffrer un fichier

Pour chiffrer un fichier avec une clé symétrique (AES, par exemple), utilisez :

```
openssl enc -aes-256-cbc -salt -in fichier.txt -out fichier.txt.enc
```

Vous serez invité à entrer une phrase de passe.

d. Déchiffrer un fichier

Pour déchiffrer le fichier, utilisez :

```
openssl enc -d -aes-256-cbc -in fichier.txt.enc -out fichier.txt
```

e. Générer une paire de clés RSA

Pour générer une paire de clés RSA :

```
openssl genpkey -algorithm RSA -out private_key.pem  
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

f. Signer et vérifier des fichiers

Pour signer un fichier :

```
openssl dgst -sha256 -sign private_key.pem -out signature.bin fichier.txt
```

Pour vérifier la signature :

```
openssl dgst -sha256 -verify public_key.pem -signature signature.bin  
fichier.txt
```

Le chiffrement des données et la gestion des clés sont essentiels pour protéger les informations sensibles. GPG est idéal pour le chiffrement de fichiers et la gestion des clés publiques, tandis qu'OpenSSL offre une flexibilité pour le chiffrement symétrique et asymétrique ainsi que pour la gestion des certificats. Utilisez ces outils pour renforcer la sécurité de vos communications et de vos données.

V. Surveillance des systèmes pour détecter les anomalies (auditd, SELinux).

La surveillance des systèmes pour détecter les anomalies est cruciale pour maintenir la sécurité et l'intégrité des systèmes Linux. Voici un aperçu des outils `auditd` et `SELinux` qui peuvent être utilisés pour cette tâche.

1. Surveillance avec auditd

a. Description

`auditd` (Audit Daemon) est un service qui collecte et stocke des événements de sécurité sur un système. Il permet de suivre les actions des utilisateurs, les modifications de fichiers et d'autres événements critiques.

b. Installation

Pour installer `auditd` sur Debian/Ubuntu :

```
bash
```

```
sudo apt install auditd
```

Sur CentOS/RHEL :

bash

```
sudo yum install audit
```

c. Configuration de auditd

Le fichier de configuration principal se trouve à `/etc/audit/auditd.conf`. Voici quelques paramètres importants :

- `log_file` : Spécifie l'emplacement du fichier journal.
- `max_log_file` : Définit la taille maximale du fichier journal avant rotation.

d. Règles d'audit

Les règles d'audit se trouvent dans le fichier `/etc/audit/audit.rules`. Voici quelques exemples de règles :

Surveiller les accès aux fichiers :

```
-w /etc/passwd -p rwx -k passwd_changes
```

Surveiller les commandes sudo :

```
-w /var/log/sudo.log -p wa -k sudo_commands
```

e. Démarrer et vérifier auditd

Pour démarrer le service :

```
sudo systemctl start auditd  
sudo systemctl enable auditd
```

Pour vérifier les événements enregistrés :

```
ausearch -k passwd_changes
```

2. Sécurisation avec SELinux

a. Description

SELinux (Security-Enhanced Linux) est une architecture de sécurité qui fournit un contrôle d'accès obligatoire (MAC). Il permet de définir des politiques de sécurité strictes pour les processus et les fichiers.

b. Installation

SELinux est généralement inclus dans les distributions basées sur Red Hat. Sur Debian/Ubuntu, il est disponible sous forme de paquet :

bash

```
sudo apt install selinux-basics selinux-policy-default
```

c. Vérifier l'état de SELinux

Pour vérifier l'état actuel de SELinux :

bash

```
sestatus
```

d. Modes de fonctionnement

SELinux peut fonctionner en trois modes :

- ✓ **Enforcing** : Applique les politiques de sécurité.
- ✓ **Permissive** : Enregistre les violations des politiques sans les bloquer.
- ✓ **Disabled** : Désactive SELinux.

Pour changer le mode (temporairement) :

```
sudo setenforce 0 # Permissif
sudo setenforce 1 # Enforcing
```

e. Configurer les politiques

Les politiques de SELinux sont définies dans `/etc/selinux/targeted/policy/`. Vous pouvez créer des règles personnalisées pour autoriser ou refuser des accès spécifiques.

f. Journalisation des événements

Les événements de SELinux sont généralement enregistrés dans `/var/log/audit/audit.log`. Vous pouvez utiliser `audit2allow` pour analyser ces journaux et générer des règles.

La surveillance des systèmes avec `auditd` et SELinux permet de détecter les anomalies et de renforcer la sécurité des systèmes Linux. `auditd` fournit une vue détaillée des événements de sécurité, tandis que SELinux impose des politiques de sécurité strictes pour contrôler l'accès aux ressources. En utilisant ces outils ensemble, vous pouvez améliorer considérablement la sécurité et la résilience de votre infrastructure Linux.

